Slide 1: Session opening

Main point: Understand which component does each job in a language-model workflow.

A language model is part of an application. The application supplies inputs, connects tools and executes permitted actions. In some workflows code prescribes the steps; in others the model requests its next step after seeing a result. We will make those roles visible before discussing training or complex governance. The practical question is whether the design supplies the right information and whether we can inspect what actually happened.

Example to use The same question about covenant headroom can be answered from a retrieved paragraph, a calculator result or a sequence of model-requested tool calls. These are different workflows even when the final prose looks similar.

Assumptions and limits The examples are fictional. The offline notebook uses a clearly identified scripted model to illustrate interfaces, not to measure a real model capability.

Ask the room Which part of your current application chooses what happens next?

Slide 2: What Section 1 adds

Main point: Explain how architecture and retrieval sessions differ.

Section 1 identifies responsibilities: the generator, application, retrieval service, calculator and controller. Section 2 then opens the retrieval service to show how information is represented, searched and ranked. We need both views. A useful architecture diagram tells us where a task happens, but does not guarantee that its implementation finds the right document. Start with the boundaries so the later technical details have a place.

Example to use We first show that a search result is supplied as model context. Next session we examine why that search returned one passage rather than another.

Assumptions and limits Architecture descriptions are design choices; a diagram alone does not verify actual execution or data access.

Ask the room Which component does the search, and which component interprets the returned text?

Slide 3: Five things you will be able to explain

Main point: Turn the objectives into practical explanations.

You should be able to draw the model boundary, distinguish retrieval from training, identify model roles, compare fixed and model-directed control and justify a design under practical limits. These are decisions you can apply without implementing a transformer. The exercises will expose a query, a source packet, a calculator call and a sequence of requested tools. The objective is to read that trace and explain its consequences.

Example to use You can point to a calculator argument and say whether code or the model selected it, then trace it back to a source.

Assumptions and limits The exercises illustrate a small controlled workflow. They do not establish that every model will choose the same sequence.

Ask the room Which of these explanations would be most useful to your team?

Slide 4: Start with the language-model boundary — 1 of 3

Main point: The model receives an input

Instructions, the question and any supplied evidence are assembled into what the model can read. The model receives instructions, a question and any supplied context. It produces text or structured tool requests. The application provides the input and decides how to handle permitted outputs. In an agentic workflow the model may choose which search or tool to request next. The application still executes the request and returns the result. Prompting the model to search is not the same as giving it an executable search connection.

Example to use The model requests calculate_leverage with debt 75, eligible cash 10 and EBITDA 20. Python performs the division and returns 3.25. The model can then interpret that result.

Assumptions and limits Tool availability, permissions, schemas and execution must be implemented by the application or tool service. A fluent description of a call is not evidence it ran.

Ask the room Who computes the division: the language model or the calculator code?

Slide 5: Start with the language-model boundary — 2 of 3

Main point: The model can request a tool

A request names a tool and its arguments. The model has not executed the function merely by requesting it. The model receives instructions, a question and any supplied context. It produces text or structured tool requests. The application provides the input and decides how to handle permitted outputs. In an agentic workflow the model may choose which search or tool to request next. The application still executes the request and returns the result. Prompting the model to search is not the same as giving it an executable search connection.

Example to use The model requests calculate_leverage with debt 75, eligible cash 10 and EBITDA 20. Python performs the division and returns 3.25. The model can then interpret that result.

Assumptions and limits Tool availability, permissions, schemas and execution must be implemented by the application or tool service. A fluent description of a call is not evidence it ran.

Ask the room Who computes the division: the language model or the calculator code?

Slide 6: Start with the language-model boundary — 3 of 3

Main point: The application executes permitted requests

Python or a connected service runs the function and returns its result. The model then continues from that result. The model receives instructions, a question and any supplied context. It produces text or structured tool requests. The application provides the input and decides how to handle permitted outputs. In an agentic workflow the model may choose which search or tool to request next. The application still executes the request and returns the result. Prompting the model to search is not the same as giving it an executable search connection.

Example to use The model requests calculate_leverage with debt 75, eligible cash 10 and EBITDA 20. Python performs the division and returns 3.25. The model can then interpret that result.

Assumptions and limits Tool availability, permissions, schemas and execution must be implemented by the application or tool service. A fluent description of a call is not evidence it ran.

Ask the room Who computes the division: the language model or the calculator code?

Slide 7: Two ways information reaches a model

Main point: Distinguish stored parameters from current input.

Training changes learned parameters. Context is supplied for a particular call and can include new documents and tool outputs. Both affect the answer, but they have different update mechanisms. Retrieval is useful when the source is current, private or too specific to expect in the weights. Further training may help recurring response behavior; it does not establish what a missing new contract says.

Example to use A newly signed notice can be retrieved today without retraining. Teaching a model to distinguish effective and superseded dates requires a separate behavioral evaluation and perhaps training.

Assumptions and limits Ordinary context augmentation does not update parameters. Product memory and storage features are separate application mechanisms.

Ask the room Which path changes when you paste a new clause into a question?

Slide 8: Different models can do different jobs — 1 of 3

Main point: Generator: write or request

A generative language model, such as GPT or Qwen, produces an answer or a structured tool request. A generator writes a response or requests tools. An embedding model maps text to a vector used for similarity search. A reranker reads a query and candidate passage and scores their compatibility. One application can use all three. They are roles rather than mutually exclusive product categories or size classes. A keyword search or database query may use no embedding model at all.

Example to use An embedding search finds twenty candidate contract paragraphs, a reranker selects the best matches and a GPT or Qwen generator drafts an answer from the selected text.

Assumptions and limits The names are examples, not a verified description of ChatGPT internals. Relevance scores are not automatically probabilities of truth.

Ask the room Does using three models automatically make a workflow agentic? No: control can still be fixed in code.

Slide 9: Different models can do different jobs — 2 of 3

Main point: Embedding model: help find candidates

It converts a query and passages into vectors that a search service compares. OpenAI embedding models are one example. A generator writes a response or requests tools. An embedding model maps text to a vector used for similarity search. A reranker reads a query and candidate passage and scores their compatibility. One application can use all three. They are roles rather than mutually exclusive product categories or size classes. A keyword search or database query may use no embedding model at all.

Example to use An embedding search finds twenty candidate contract paragraphs, a reranker selects the best matches and a GPT or Qwen generator drafts an answer from the selected text.

Assumptions and limits The names are examples, not a verified description of ChatGPT internals. Relevance scores are not automatically probabilities of truth.

Ask the room Does using three models automatically make a workflow agentic? No: control can still be fixed in code.

Slide 10: Different models can do different jobs — 3 of 3

Main point: Reranker: compare the shortlist

A cross-encoder, such as a BGE reranker, scores each query–passage pair. It reorders candidates before generation. A generator writes a response or requests tools. An embedding model maps text to a vector used for similarity search. A reranker reads a query and candidate passage and scores their compatibility. One application can use all three. They are roles rather than mutually exclusive product categories or size classes. A keyword search or database query may use no embedding model at all.

Example to use An embedding search finds twenty candidate contract paragraphs, a reranker selects the best matches and a GPT or Qwen generator drafts an answer from the selected text.

Assumptions and limits The names are examples, not a verified description of ChatGPT internals. Relevance scores are not automatically probabilities of truth.

Ask the room Does using three models automatically make a workflow agentic? No: control can still be fixed in code.

Slide 11: Separate three architecture choices

Main point: Keep model, hosting and control flow independent.

Choosing a model addresses capabilities and resource needs. Choosing hosting determines where computation runs and which service policies apply. Choosing control flow determines whether the application prescribes the sequence or lets the model request next steps. Do not bundle these decisions into one label. An open-weight model can run in a cloud account, and a small model can be offered through an external API.

Example to use A compact model hosted privately can run an agentic tool loop, while a large hosted model can sit at the end of a completely fixed pipeline.

Assumptions and limits Hosting does not by itself establish privacy or security. Data flow, configuration and policy compatibility need separate checks.

Ask the room Which of the three choices is actually constrained in your organization?

Slide 12: Choose the operating limitations first

Main point: Motivate hosting and model choice with real constraints.

Begin with the boundaries the application must respect. Consider whether the data may leave a controlled environment, where processing and storage occur, how long service data is retained and which security policies apply. Then consider latency, hardware and operational skills. Region matters when organizational, contractual or legal requirements constrain data location; retention concerns what data and logs a service stores and for how long.

Example to use A team may be allowed to use an external API for public filings but required to keep client records on controlled infrastructure.

Assumptions and limits Provider terms and service settings vary and must be verified. A private endpoint or local deployment alone is not a complete security assessment.

Ask the room What data-policy requirement would rule out an otherwise capable service?

Slide 13: Example: a new contract motivates retrieval

Main point: Show why current information belongs in context.

This is an example of an information problem. A contract is signed after the model was trained or is specific to the organization. The application must obtain the controlling text and give the relevant parts to the model. Increasing model size or asking it to remember harder does not establish those terms. Retrieval connects an external source to answer generation. We will still need to verify that it retrieves the right version and sufficient qualifications.

Example to use The notice period changes from thirty days to sixty days in a new agreement. A model relying on old knowledge can be articulate and wrong.

Assumptions and limits The example does not assume the learned weights contain no contracts; it identifies why they cannot be relied on for this particular current document.

Ask the room What evidence would show which agreement actually governed the answer?

Slide 14: RAG makes a source part of the model input

Main point: Explain retrieval-augmented generation as a whole pattern.

The search tool receives a query and returns passages with source identifiers. The application places the selected text beside the question and instructions in the model input. The model generates an answer using that input. This complete pattern is retrieval-augmented generation, or RAG. RAG is not a single component that understands every source, chooses every parser and certifies the result. Each responsibility must be assigned.

Example to use Query notice period returns a paragraph saying cancellation requires thirty days. The application includes that paragraph in the prompt so the model can answer the notice-period question.

Assumptions and limits Retrieval can miss, misrank or return outdated material. Context inclusion does not guarantee faithful use.

Ask the room Which arrow actually puts the source text into what the model can read?

Slide 15: See the actual context supplied to the model

Main point: Make augmentation visible rather than leave it as a slogan.

Read the assembled input as three parts: task instructions, the original question and selected source text. The generator sees what the application actually sends, not everything in the document collection. Recording only search results is insufficient if a later selection step discards or truncates them. Inspecting the final input is how we establish the answer-time information boundary.

Example to use A search can retrieve a relevant paragraph but the prompt can omit its exception sentence. The retriever may have succeeded while context construction failed.

Assumptions and limits Source IDs and versions must identify the actual supplied text. A source being available in a database is not the same as its passage being in context.

Ask the room What could be lost between search output and the final model input?

Slide 16: RAG is not the same as vector search — 1 of 3

Main point: Retrieval finds information

A search tool can return text, table rows or API results. The application chooses and connects the permitted sources. Vector search compares learned text representations. RAG is the larger pattern of retrieving information and supplying it for generation. Retrieval can use exact keywords, a database query, a spreadsheet reader, an API or a combination. The tool implementation determines what is searchable and how results are produced. The model may request a query in an agentic workflow, but it does not automatically configure every data source.

Example to use A SQL query can return a debt and cash table, while a document search returns the covenant definition. Both results can become context for the same generated answer.

Assumptions and limits Structured tools need their own schemas, access restrictions and validation. Vector proximity is not an eligibility or truth check.

Ask the room Would a fixed SQL lookup followed by a generated summary still be a retrieval-augmented workflow?

Slide 17: RAG is not the same as vector search — 2 of 3

Main point: Augmentation puts results in context

The application supplies selected results with the original question. Being in a database is not the same as reaching the model. Vector search compares learned text representations. RAG is the larger pattern of retrieving information and supplying it for generation. Retrieval can use exact keywords, a database query, a spreadsheet reader, an API or a combination. The tool implementation determines what is searchable and how results are produced. The model may request a query in an agentic workflow, but it does not automatically configure every data source.

Example to use A SQL query can return a debt and cash table, while a document search returns the covenant definition. Both results can become context for the same generated answer.

Assumptions and limits Structured tools need their own schemas, access restrictions and validation. Vector proximity is not an eligibility or truth check.

Ask the room Would a fixed SQL lookup followed by a generated summary still be a retrieval-augmented workflow?

Slide 18: RAG is not the same as vector search — 3 of 3

Main point: Generation produces the answer

The language model interprets the supplied context. Vector search is one possible retrieval method, not the whole RAG workflow. Vector search compares learned text representations. RAG is the larger pattern of retrieving information and supplying it for generation. Retrieval can use exact keywords, a database query, a spreadsheet reader, an API or a combination. The tool implementation determines what is searchable and how results are produced. The model may request a query in an agentic workflow, but it does not automatically configure every data source.

Example to use A SQL query can return a debt and cash table, while a document search returns the covenant definition. Both results can become context for the same generated answer.

Assumptions and limits Structured tools need their own schemas, access restrictions and validation. Vector proximity is not an eligibility or truth check.

Ask the room Would a fixed SQL lookup followed by a generated summary still be a retrieval-augmented workflow?

Slide 19: Fixed RAG versus agentic RAG

Main point: Explain the difference in who selects subsequent steps.

A fixed pipeline runs a prescribed retrieval and generation sequence. An agentic loop gives the model tools and lets it request another step after inspecting a result. Both rely on application execution. A model-generated search query is not by itself a full agent; the important distinction is whether subsequent tool use and sequence are selected dynamically. Fixed code can also include branches and retries.

Example to use A retrieved clause references Annex A. The fixed pipeline shown may stop, while the model-directed loop can request the annex. A programmed branch could also do that retrieval.

Assumptions and limits The distinction concerns control flow, not guaranteed quality or capability. Agentic systems can skip needed steps and fixed workflows can be sophisticated.

Ask the room Which part of the displayed trace reveals who chose the follow-up search?

Slide 20: Which component chooses the next step? — 1 of 3

Main point: Fixed sequence: code chooses

The application always retrieves, calculates and then asks the model to interpret. The model does not choose those earlier steps. A fixed pipeline runs a prescribed retrieval and generation sequence. An agentic loop gives the model tools and lets it request another step after inspecting a result. Both rely on application execution. A model-generated search query is not by itself a full agent; the important distinction is whether subsequent tool use and sequence are selected dynamically. Fixed code can also include branches and retries.

Example to use A retrieved clause references Annex A. The fixed pipeline shown may stop, while the model-directed loop can request the annex. A programmed branch could also do that retrieval.

Assumptions and limits The distinction concerns control flow, not guaranteed quality or capability. Agentic systems can skip needed steps and fixed workflows can be sophisticated.

Ask the room Which part of the displayed trace reveals who chose the follow-up search?

Slide 21: Which component chooses the next step? — 2 of 3

Main point: Agent loop: the model requests

The model sees a result and can request a follow-up search or calculation. The application executes each permitted request. A fixed pipeline runs a prescribed retrieval and generation sequence. An agentic loop gives the model tools and lets it request another step after inspecting a result. Both rely on application execution. A model-generated search query is not by itself a full agent; the important distinction is whether subsequent tool use and sequence are selected dynamically. Fixed code can also include branches and retries.

Example to use A retrieved clause references Annex A. The fixed pipeline shown may stop, while the model-directed loop can request the annex. A programmed branch could also do that retrieval.

Assumptions and limits The distinction concerns control flow, not guaranteed quality or capability. Agentic systems can skip needed steps and fixed workflows can be sophisticated.

Ask the room Which part of the displayed trace reveals who chose the follow-up search?

Slide 22: Which component chooses the next step? — 3 of 3

Main point: Either can follow an annex reference

Code can include a programmed branch. The choice is prescribed branching versus model-selected next steps, not tools versus no tools. A fixed pipeline runs a prescribed retrieval and generation sequence. An agentic loop gives the model tools and lets it request another step after inspecting a result. Both rely on application execution. A model-generated search query is not by itself a full agent; the important distinction is whether subsequent tool use and sequence are selected dynamically. Fixed code can also include branches and retries.

Example to use A retrieved clause references Annex A. The fixed pipeline shown may stop, while the model-directed loop can request the annex. A programmed branch could also do that retrieval.

Assumptions and limits The distinction concerns control flow, not guaranteed quality or capability. Agentic systems can skip needed steps and fixed workflows can be sophisticated.

Ask the room Which part of the displayed trace reveals who chose the follow-up search?

Slide 23: Question: is this workflow agentic?

Main point: Check the difference between using tools and choosing tool steps.

The scenario has retrieval and a calculator at predefined places. The model then receives the results and writes an answer. Tools are present, but the sequence is selected by application code. Ask participants to explain their choice using the execution trace, rather than assume any tool use makes the application agentic.

Example to use A pipeline always retrieves documents, calculates leverage and then asks for interpretation. The model need not choose or request either earlier step.

Assumptions and limits Definitions vary at the edges, but this session uses control of the next action as its practical distinction.

Ask the room Who chose when the calculator ran?

Slide 24: Answer: the application controls this sequence

Main point: Make the fixed-workflow classification explicit.

The application chose the retrieval and calculation steps. The model interpreted their outputs. That can be an appropriate architecture when the required work is known. A fixed sequence is not less intelligent in a business sense; it can make execution easier to inspect. Agentic control is useful when the next information need depends on what has been found and cannot conveniently be prescribed.

Example to use The same calculator can appear in both architectures. What changes is whether code calls it unconditionally or the model requests a call.

Assumptions and limits A fixed workflow can include conditionals and still be application-directed. The label does not settle which design performs better.

Ask the room When would a model-chosen follow-up be worth the additional variability?

Slide 25: Harbor Metals: introduce the credit question

Main point: Define the fictional case before showing its workflow.

Harbor Metals reports debt of seventy-five million, cash of fifteen million and EBITDA of twenty million. The covenant ceiling is three and a half times net leverage. Our question is whether headroom is comfortable. First establish the contractual calculation and current compliance. Comfort is a separate judgment requiring a defined buffer, downside assumptions and an accountable interpretation. Do not let the adjective silently become an objective numerical fact.

Example to use Using all cash gives three times leverage, but the covenant excludes restricted cash. The annex identifies five million restricted, giving eligible cash of ten and leverage of 3.25.

Assumptions and limits All values are fictional and in matching monetary units. The definition of eligible cash is supplied by the covenant. Comfort has no universal threshold here.

Ask the room What additional assumption would you need before calling the buffer comfortable?

Slide 26: The retrieval output must reach the model

Main point: Distinguish available documents from supplied passages.

The initial packet contains a filing and covenant. The covenant refers to an annex that supplies a needed qualification. The application must transmit the retrieved text, not just say that documents exist. If the annex is not returned or passed along, the model cannot establish the eligible-cash value from this packet. We will inspect that handoff rather than infer support from a polished answer.

Example to use A filing reports cash fifteen, while the annex says five is restricted. Leaving out the annex changes the supported calculator input.

Assumptions and limits The collection, retrieval request and final context are separate objects. A missing passage can reflect scope, search or context selection, not necessarily an absent file.

Ask the room Where would you look to see whether the annex was actually supplied?

Slide 27: Where retrieval and calculation fit

Main point: Show how sources and a numerical tool contribute different things.

Retrieval supplies source statements. The model reads that context and may request a calculator with selected inputs. The tool executes arithmetic and returns its result. The model then writes an answer. The term RAG names the retrieval-and-generation pattern around those steps, not a hidden extra box. A correct division does not validate the source values used in it.

Example to use The calculator receives debt seventy-five, eligible cash ten and EBITDA twenty and returns 3.25. The annex supports ten; the calculator does not discover that amount itself.

Assumptions and limits Arithmetic assumes the formula and inputs match the contractual definition and units. Neither the tool call nor its result proves that headroom is comfortable.

Ask the room Which component checks arithmetic, and what establishes the cash input?

Slide 28: Fixed workflow: code specifies the required steps

Main point: Explain the application-directed version of the case.

The application retrieves the documents, extracts fields and calls the calculator before asking the model for an interpretation. Each step is visible in code. A simplistic extractor may use reported rather than eligible cash, so even a fixed workflow needs correct parsing and dependency handling. The example illustrates how to inspect the handoffs, not a claim that fixed workflows must make this error.

Example to use Using cash fifteen gives three times leverage. Adding a programmed annex step and subtracting restricted cash produces 3.25.

Assumptions and limits Extraction logic and source scope are implementation choices. The generator cannot execute an extra request unless the application enables a tool loop or branch.

Ask the room Which line of code would you change to ensure the annex is considered?

Slide 29: Agent workflow: the result changes the next request

Main point: Explain observation-dependent tool selection.

The model starts with a question and tool descriptions. It requests retrieval, sees an annex reference and can request a follow-up. After reading the result it chooses calculator arguments. The application executes each permitted request and returns outputs. This flexibility adds choices that must be evaluated: whether to retrieve again, which query to use, when to calculate and when to stop.

Example to use The observation Annex A defines restricted cash motivates a new search. That next request depends on what the first result revealed.

Assumptions and limits The illustrative offline model is scripted; live models can skip calls, choose poor queries or answer too early. A tool request in prose is not an executed call.

Ask the room What observation should trigger the annex search?

Slide 30: Fixed code can follow a programmed branch too

Main point: Prevent the false conclusion that only an agent can retrieve an annex.

Code can implement an explicit branch: if a returned contract references an annex, retrieve it. That remains a fixed application-directed workflow because the branch logic was prescribed. A model-directed agent chooses its next request within allowed tools based on its observations. Choose between them by the variability of the task and the cost of possible mistakes, not by treating agentic as an automatic upgrade.

Example to use A known family of contracts always places cash exclusions in Annex A. A programmed rule may be simple and reliable. More varied documents may motivate model-assisted follow-up.

Assumptions and limits A branch needs correct reference detection and version binding. Model-directed discovery also needs those source checks and a bounded execution loop.

Ask the room Is the next step sufficiently known that you can specify it directly?

Slide 31: Harbor Metals: connect the model to two functions

Main point: Show what makes tools executable rather than merely described.

The application exposes search_documents and calculate_leverage with input definitions and descriptions. The prompt explains the task and asks the model to follow annex references. The tool interface tells it how a request must be structured. A runtime dispatches the request to actual code. These are three different pieces: instructions, callable interface and execution.

Example to use search_documents takes a query and returns passages with source IDs. calculate_leverage takes three numbers and returns a ratio.

Assumptions and limits Schemas constrain request format, not evidence correctness. The application must validate tool names and inputs and enforce permissions.

Ask the room Would putting the function name in a prompt alone make it run?

Slide 32: Direct function calling: request, execute, return

Main point: Walk through the two arrows in the tool loop.

A function-calling model emits a structured tool name and arguments. Python receives that request, checks it against allowed functions and executes the corresponding code. The result is returned with the call identifier so the model can continue. The application repeats until the model answers or a budget is reached. This makes the execution trace visible. MCP is not required for one application calling its own functions.

Example to use Request calculator with debt seventy-five, eligible cash ten, EBITDA twenty; application returns 3.25; model uses or fails to use that result in its final response.

Assumptions and limits The loop needs limits, errors and permission handling. A successful request does not guarantee the final answer agrees with the returned result.

Ask the room What would show that a model skipped calculation rather than merely omitted it from the explanation?

Slide 33: When MCP helps: reuse Harbor Metals tools

Main point: Motivate MCP through shared integration rather than multiple agents alone.

MCP is a standard interface for exposing and discovering tools. A credit assistant, monitoring workflow and notebook can use a compatible server instead of each needing a custom integration to the backend. Their workflows can remain different. MCP does not coordinate them, choose their plans or authorize every request automatically. Its value is reusable tool connectivity.

Example to use The document-search implementation changes behind the server while the search_documents interface stays stable. Compatible clients can keep their existing connection.

Assumptions and limits The local lab launches a subprocess MCP server for demonstration. A production shared service requires authenticated identities, policy enforcement and operational controls.

Ask the room What would be reused across the three applications, and what would remain application-specific?

Slide 34: Permission to calculate is not permission to release

Main point: Separate tool access from action authority.

The research role may search and calculate but may not release a brief. A reviewer role can release while its grant remains valid. Revoking that grant blocks the same request. The check must run in the application or service before the side effect. A prompt asking the model to behave is not a substitute for permission enforcement. This is the introduction we will revisit in governance.

Example to use The notebook blocks a research release, permits one simulated reviewer release and blocks a later attempt after revocation.

Assumptions and limits The demo trusts local launcher-supplied role names. Production must derive identity from authentication, protect grants and avoid bypasses. Authorization does not certify the brief is true.

Ask the room If the model asks for release after revocation, what component must stop it?

Slide 35: Class exercise: Harbor Metals tools and permissions

Main point: Guide the concrete sequence participants should inspect.

Open the companion notebook and run the local path first. Search the filing, identify the annex reference, retrieve the annex and inspect the calculator arguments. Then attempt a simulated release under research, reviewer and revoked reviewer conditions. The optional GPT path shows model-chosen calls; the optional MCP path shows tool discovery and shared access. The browser option is Colab opened from GitHub and saved to Drive.

Example to use Point to the returned source ID, then the cash argument ten, then the 3.25 result, and finally the recorded permission denial.

Assumptions and limits Default exercises run without a key. Live provider calls can behave differently and incur charges. Only fictional data should be used in the public exercise.

Ask the room Ask each participant to explain one denial or one source-to-tool handoff.

Slide 36: Retrieval and training solve different problems — 1 of 3

Main point: A required fact is missing

Obtain the current source and supply it as context. Training on old examples cannot establish what an unavailable new document says. Retrieval supplies information at answer time. Further training changes learned behavior. If the latest source is absent, first repair access or search. If correct evidence is supplied but the model repeatedly confuses actual and forecast, improve instructions and consider reviewed training examples. Evaluate any update on separate unseen cases. Performance on the training examples only shows fit to those examples.

Example to use Train with records distinguishing reported results from expected results, then test on different wording and companies. Do not reuse the same records to certify improvement.

Assumptions and limits The cause must be diagnosed first. Training does not create a missing authoritative source or guarantee calibration.

Ask the room Which repair would address an obsolete agreement in the prompt?

Slide 37: Retrieval and training solve different problems — 2 of 3

Main point: The source is present but repeatedly misread

Improve instructions. If errors persist, consider fine-tuning on reviewed demonstrations of the required behavior. Retrieval supplies information at answer time. Further training changes learned behavior. If the latest source is absent, first repair access or search. If correct evidence is supplied but the model repeatedly confuses actual and forecast, improve instructions and consider reviewed training examples. Evaluate any update on separate unseen cases. Performance on the training examples only shows fit to those examples.

Example to use Train with records distinguishing reported results from expected results, then test on different wording and companies. Do not reuse the same records to certify improvement.

Assumptions and limits The cause must be diagnosed first. Training does not create a missing authoritative source or guarantee calibration.

Ask the room Which repair would address an obsolete agreement in the prompt?

Slide 38: Retrieval and training solve different problems — 3 of 3

Main point: An update appears to help

Evaluate on separate unseen cases. Better performance on the training examples alone does not establish generalization. Retrieval supplies information at answer time. Further training changes learned behavior. If the latest source is absent, first repair access or search. If correct evidence is supplied but the model repeatedly confuses actual and forecast, improve instructions and consider reviewed training examples. Evaluate any update on separate unseen cases. Performance on the training examples only shows fit to those examples.

Example to use Train with records distinguishing reported results from expected results, then test on different wording and companies. Do not reuse the same records to certify improvement.

Assumptions and limits The cause must be diagnosed first. Training does not create a missing authoritative source or guarantee calibration.

Ask the room Which repair would address an obsolete agreement in the prompt?

Slide 39: Class exercise: inspect the handoffs

Main point: Make the fixed-versus-agent comparison observable.

Run the same question under fixed retrieval, fixed retrieval plus calculator and the agent loop. Inspect the exact query and the passages returned. Find the function call and its arguments. In the agent condition follow the requested tool sequence. Add an annex branch to the fixed workflow and compare again. The important observation is not which answer sounds better but which steps and evidence support it.

Example to use The initial fixed calculation can use cash fifteen and report three times. A corrected fixed branch and the scripted agent can both use eligible cash ten and return 3.25.

Assumptions and limits These are controlled illustrative configurations. A recorded or scripted success is not a measured guarantee for live models.

Ask the room What change made the supported ratio change?

Slide 40: Question: what should change first?

Main point: Check whether an information failure is being mistaken for model weakness.

The answer is based on an outdated document. Before selecting a larger model or training it, inspect which source version reached the context. A more capable generator can faithfully interpret the wrong agreement and still return an inapplicable answer. Ask for the evidence needed to distinguish source selection from interpretation.

Example to use An old contract has a thirty-day notice period and the current one sixty. The source version determines which answer is justified.

Assumptions and limits The question assumes the outdated-source diagnosis is established. If sources conflict, controlling status may require domain review.

Ask the room Which log field would show the exact source version used?

Slide 41: Answer: inspect the selected source version

Main point: Explain why the first repair belongs in the evidence path.

Identify the controlling source, its effective date and the actual text supplied to the model. Fix collection, filtering or context construction as needed, then rerun the affected case. Only after valid evidence is supplied can we judge whether the generator still misinterprets it. This order avoids training around an information problem.

Example to use Replace the obsolete thirty-day clause with the current sixty-day clause and check whether the answer follows it with the relevant qualifications.

Assumptions and limits An available timestamp is not always an effective date. Applicable source rules must be defined for the task.

Ask the room What would you test next if the model still says thirty after receiving the correct clause?

Slide 42: Architecture choices involve practical trade-offs — 1 of 3

Main point: Check operating constraints

Data policy, security, permitted processing locations and retention requirements may rule out an otherwise capable service. Compare cost, latency, reliability, memory, hardware and staff skills across the complete workload. A model-directed loop may need several provider calls and variable tool latency. A fixed pipeline may be easier to inspect when steps are known. Local hosting can improve control over data flow but requires capacity and operational responsibility. Measure the actual workflow rather than only the model token price.

Example to use A cheaper per-token model that performs six searches and repeated retries can cost more per completed task than a larger model in a stable fixed pipeline.

Assumptions and limits Costs and capabilities change over time. Reliable comparisons need current measurements, matched tasks and realistic failure handling.

Ask the room Which constraint is binding: data policy, response time, memory or operational skills?

Slide 43: Architecture choices involve practical trade-offs — 2 of 3

Main point: Measure the complete workload

Count provider calls, searches, calculations, retries and latency. Token price alone is not the cost of a completed task. Compare cost, latency, reliability, memory, hardware and staff skills across the complete workload. A model-directed loop may need several provider calls and variable tool latency. A fixed pipeline may be easier to inspect when steps are known. Local hosting can improve control over data flow but requires capacity and operational responsibility. Measure the actual workflow rather than only the model token price.

Example to use A cheaper per-token model that performs six searches and repeated retries can cost more per completed task than a larger model in a stable fixed pipeline.

Assumptions and limits Costs and capabilities change over time. Reliable comparisons need current measurements, matched tasks and realistic failure handling.

Ask the room Which constraint is binding: data policy, response time, memory or operational skills?

Slide 44: Architecture choices involve practical trade-offs — 3 of 3

Main point: Match resources and skills

Local hosting needs hardware and operational support. Choose the simplest adequate workflow that meets the measured requirements. Compare cost, latency, reliability, memory, hardware and staff skills across the complete workload. A model-directed loop may need several provider calls and variable tool latency. A fixed pipeline may be easier to inspect when steps are known. Local hosting can improve control over data flow but requires capacity and operational responsibility. Measure the actual workflow rather than only the model token price.

Example to use A cheaper per-token model that performs six searches and repeated retries can cost more per completed task than a larger model in a stable fixed pipeline.

Assumptions and limits Costs and capabilities change over time. Reliable comparisons need current measurements, matched tasks and realistic failure handling.

Ask the room Which constraint is binding: data policy, response time, memory or operational skills?

Slide 45: Section 1: objectives revisited

Main point: Check whether participants can explain a complete path in plain language.

Ask someone to trace a question through retrieval, application context construction, model interpretation, tool request, execution and final response. Ask another person to say which steps are fixed and which are chosen after observations. A clear explanation identifies responsibilities without treating RAG or agent as a magical component. Section 2 now opens the retrieval box to inspect relevance, ranking and source selection.

Example to use Explain why having Annex A in the collection is not the same as sending its text to the model.

Assumptions and limits The review establishes understanding, not empirical model qualification. Live behavior still needs testing.

Ask the room Which retrieval detail do you need to inspect before trusting the architecture?